

以下是几道经典的树上动态规划题目及对应的解题思路与程序框架：

一、二叉苹果树

题目描述：

有一棵二叉苹果树，如果数字有分叉，一定是分两叉，即没有只有一个儿子的节点。这棵树共 N 个节点，标号 1 至 N ，树根编号一定为 1 。我们用一根树枝两端连接的节点编号描述一根树枝的位置。一棵有四根树枝的苹果树，因为树枝太多了，需要剪枝。但是一些树枝上长有苹果，给定需要保留的树枝数量，求最多能留住多少苹果。

解题思路：

相当于从一棵二叉树上去掉一些子树，使得剩下 m 条边，目标是使得这 m 条边的总权值最大。剩 m 条边就相当于剩 $m+1$ 个点，可以转换成点来做。设 $f[i,j]$ 表示节点 i 还剩 j 个点时最大总权值，目标状态是 $f[1,m+1]$ 。可以利用记忆化搜索来求解。

程序框架：

cpp

Copy Code

```
#include <bits/stdc++.h>
using namespace std;

const int N = 110;
```

```
const int M = 440;
int head[N], edge[M], nex[M], ver[M], tot = 1;
int n, m, f[N][N];
int ls[N], rs[N], w[N];
```

```
void addEdge(int x, int y, int z) {
    ver[++tot] = y;
    edge[tot] = z;
    nex[tot] = head[x];
    head[x] = tot;
}
```

```
void dfs(int x, int fa) {
    for (int i = head[x]; i; i = nex[i]) {
        int y = ver[i], z = edge[i];
        if (y == fa) continue;
        if (!ls[x]) rs[x] = y;
        else ls[x] = y;
        w[y] = z; // y到其父亲的边权
        dfs(y, x);
    }
}
```

```
int DP(int i, int j) {
    if (!j) return 0;
    if (!ls[i] && !rs[i]) return w[i];
    if (f[i][j] > 0) return f[i][j];
    for (int k = 0; k <= j - 1; k++)
        f[i][j] = max(f[i][j], DP(ls[i], k) + DP(rs[i], j - k - 1) +
```

```

w[i]);
    return f[i][j];
}

int main() {
    scanf("%d%d", &n, &m);
    for (int i = 1, x, y, z; i <= n; i++) {
        scanf("%d%d%d", &x, &y, &z);
        addEdge(x, y, z);
        addEdge(y, x, z);
    }
    dfs(1, 0);
    printf("%d\n", DP(1, m + 1)); // 1号子树保留m+1
    个节点时最大值
    return 0;
}

```

二、选课

题目描述：

大学实行学分制，假定课程间不存在时间上的冲突。现在要求计算在给定的选课机会下，能获得的最高学分。

解题思路：

这题本质上是有树形依赖的分组背包问题。首先用0号虚拟节点将森林合并为一棵树，注意0号节点不算一门课，没有学分也不消耗选课机会。设 $f[x, j]$ 表示在

以 x 为根的子树中选 j 门课程能得到的最大学分。那么 $f[x,j]$ 就可以表示为“在以 x 为根的子树上选 j 门，在子树 y 上选 k 门”的最大值。

程序框架：

由于篇幅限制，此处不提供完整代码，但核心思路是使用动态规划，通过递归或迭代的方式计算每个子树中的最优解，并最终合并得到整棵树的最优解。

三、战略游戏

题目描述：

Bob喜欢玩电脑游戏，特别是战略游戏。现在他有个问题：他有一座古城堡，古城堡的路形成一棵树。他要在这棵树的节点上放置最少数目的士兵，使得这些士兵能够瞭望到所有的路。

解题思路：

这种题是“树的最大独立集”问题，也就是要求树上每一条边都至少有一个端点在集合内。可以使用动态规划来解决，定义 $d(i)$ 表示以 i 为根的子树中，放置士兵的最小数量。然后分别考虑选 i 和不选 i 两种情况，通过递归计算子树的最优解，并最终合并得到整棵树的最优解。

程序框架：

同样由于篇幅限制，此处不提供完整代码，但核心思路是使用动态规划，通过递归计算每个子树的最优

解，并根据题目要求合并得到整棵树的最优解。

总结

以上三道题目都是树上动态规划的经典题目，它们分别涉及到了树的剪枝、分组背包和最大独立集等问题。通过解决这些问题，可以加深对树上动态规划的理解和掌握。在实际编程中，需要注意建树、状态表示和状态转移等关键步骤，以确保算法的正确性和高效性。